# Additive Log-Fidelity Quantum k-Means for Hybrid Product Quantization

Christian Jesinghaus
Leibniz University Hannover
Hannover, Germany
jesinghaus@stud.uni-hannover.de

Jan S. Rellermeyer
Leibniz University Hannover
Hannover, Germany
rellermeyer@vss.uni-hannover.de

***Abstract*—Product quantization (PQ) requires additive blockwise scores, whereas fidelity over product states is multiplicative. In this paper, we propose applying a negative logarithm to obtain an additive score that preserves the ranking induced by global product-state fidelity and is therefore compatible with PQ lookup tables. Based on this observation, we introduce additive log-fidelity quantum k-means for hybrid product quantization and pair it with a safeguarded unit-sphere centroid update that accepts a Rayleigh–Ritz proposal when it decreases the true cluster loss and otherwise falls back to projected gradient descent. Furthermore we evaluate the resulting PQ-kNN pipeline on Digits and downsampled Fashion-MNIST, and study sign ambiguity on a synthetic signed benchmark. On Digits, the shot-based variant remains close to classical PQ. On Fashion-MNIST $8 \times 8$, the gap is larger. On the synthetic benchmark, sign-aware encoding removes the ambiguity of raw fidelity. The contribution is structural rather than superiority-based: it shows how fidelity-based clustering can be made compatible with additive PQ training and trained stably enough to yield usable codebooks on small low-dimensional benchmarks.**

***Index Terms*—quantum machine learning, clustering, product quantization, hybrid quantum-classical algorithms, approximate nearest neighbors, fidelity-based learning, logarithmic fidelity**

*Code:* https://github.com/ChristianJesinghaus/Additive-Log-Fidelity-Quantum-k-Means-for-Hybrid-Product-Quantization

## I. Introduction

Product quantization (PQ) is a standard compression technique for approximate nearest-neighbor search [1], [2]. It splits each vector into blocks, learns one codebook per block, and accumulates blockwise distortion terms during querying [1]. Because of this structure, any clustering objective used to train PQ codebooks must decompose additively across partitions.

This creates a difficulty in the quantum setting. For amplitude-encoded states, fidelity is a natural similarity measure, but for product states it factorizes multiplicatively. A direct fidelity objective therefore does not fit the additive blockwise structure on which PQ depends.

Prior work on quantum clustering and quantum $k$-means has studied quantum assignment rules, Euclidean formulations, and improved theoretical guarantees [3], [4], [5], [6], [7]. In this work, we consider a more focused problem: can a fidelity-based clustering rule be reformulated so that it is compatible with PQ while being stable enough to learn useful codebooks in a hybrid pipeline.

The key observation is that taking negative logarithms turns multiplicative fidelities into additive terms. This yields a PQ-compatible blockwise objective. That objective is then paired with a safeguarded centroid update on the unit sphere and the resulting method is then evaluated in a PQ-kNN setting. The focus here is methodological and inspects whether the formulation works well enough to support compressed nearest-neighbor retrieval and classification.

This paper makes three contributions:

1) an additive log-fidelity objective for PQ that is consistent with the ranking induced by global product-state fidelity, together with a smoothed zero-normalized variant used in practice;
2) a safeguarded centroid update that combines a Rayleigh–Ritz proposal with a projected-gradient fallback and accepts only loss-decreasing updates;
3) a proof-of-concept evaluation in a hybrid PQ-kNN pipeline on Digits, downsampled Fashion-MNIST, and a synthetic signed benchmark used to study sign-aware encoding.

## II. Related Work and Positioning

Classical product quantization is built around additive blockwise distortions: query evaluation sums one contribution from each partition [1]. Any clustering objective used to learn PQ codebooks therefore has to respect the same additive structure.

Raw fidelity does not. For product states, the total fidelity is the product of the block fidelities rather than their sum. This is the specific technical mismatch addressed in this paper.

Therefore, our method differs from many of the other quantum $k$-means research works in that much of that literature starts from Euclidean distance, focuses on assignment primitives, or studies algorithmic speedups and theoretical guarantees [3], [4], [5], [6], [7]. Here the starting point is a concrete downstream application. The question is whether a fidelity-based objective can be rewritten so that it fits product quantization and still yields useful codebooks in a hybrid pipeline and what needs to be build around it in order to work with that new objective.

## III. Method: Additive Log-Fidelity Quantum k-Means

### A. Problem Setup

Let $X = \{x_i\}_{i=1}^N \subset \mathbb{R}^D$ be a dataset of real-valued vectors. Product quantization partitions each vector into $m$ blocks,

$$x_i = \big(x_i^{(1)}, x_i^{(2)}, \ldots, x_i^{(m)}\big), \qquad x_i^{(p)} \in \mathbb{R}^{d_p}, \qquad \sum_{p=1}^{m} d_p = D. \tag{1}$$

For each partition $p$, the goal is to learn a codebook

$$C^{(p)} = \big\{c_1^{(p)}, c_2^{(p)}, \ldots, c_K^{(p)}\big\}, \tag{2}$$

with centroids constrained to lie on the unit sphere in the corresponding partition space. A database vector is then represented by the tuple of its partition-wise code assignments.

The resulting codebooks are evaluated in the PQ-kNN pipeline. During training, one codebook is learned per partition. During compression, each database vector is replaced by its tuple of code indices. At query time, distances between the query blocks and the selected codewords are accumulated across partitions and used for approximate nearest-neighbor retrieval or majority-vote classification.

The clustering rule operates on normalized block representations. For a nonzero block $z \in \mathbb{R}^d$, we define

$$\psi(z) = \frac{z}{\|z\|_2}, \tag{3}$$

and interpret $\psi(z)$ as a real amplitude vector. In the real-valued setting considered here, the blockwise fidelity between two unit vectors $u, v \in \mathbb{R}^d$ is the standard pure-state fidelity [8] is:

$$F(u, v) = |\langle u, v\rangle|^2 = (u^\top v)^2. \tag{4}$$

Zero blocks are handled explicitly in the implementation so that normalization remains well defined.

For an exploratory extension, a sign-aware map is considered as well

$$\phi(z) = \big[z^+, z^-\big], \qquad z^+ = \max(z, 0), \qquad z^- = \max(-z, 0), \tag{5}$$

followed by normalization. This variant is used only in the signed synthetic study in Section V.

### B. PQ-Compatible Objective

PQ requires an additive distortion across blocks. Raw fidelity is inconvenient in this setting because fidelities of product states multiply across partitions. If we write the blockwise representation of $x_i$ as

$$\Psi(x_i) = \bigotimes_{p=1}^{m} \psi\big(x_i^{(p)}\big), \tag{6}$$

and similarly for a codeword representative

$$\Psi(c) = \bigotimes_{p=1}^{m} c^{(p)}, \tag{7}$$

then the ideal fidelity factorizes as

$$F\big(\Psi(x_i), \Psi(c)\big) = \prod_{p=1}^{m} F\big(\psi(x_i^{(p)}), c^{(p)}\big). \tag{8}$$

Applying a negative logarithm yields

$$-\log F\big(\Psi(x_i), \Psi(c)\big) = \sum_{p=1}^{m} -\log F\big(\psi(x_i^{(p)}), c^{(p)}\big), \tag{9}$$

which matches the additive structure required by PQ.

*a) Why negative log fidelity:* For product states, maximizing the total fidelity $\prod_{p=1}^m F_p$ is equivalent to minimizing $-\sum_{p=1}^m \log F_p$, because $-\log(\cdot)$ is strictly monotone. Hence the additive log-fidelity objective preserves the ranking induced by global product-state fidelity while matching the additive structure required by PQ. By contrast, additive surrogates such as $\sum_{p=1}^m (1 - F_p)$ need not preserve that ranking. For example, block fidelities $(1, 0.25)$ and $(0.55, 0.55)$ yield total fidelities $0.25$ and $0.3025$, respectively, so the latter is globally better, whereas the additive one-minus-fidelity scores are $0.75$ and $0.90$ and therefore reverse the preference. This is why the study treats log-fidelity as structurally well motivated even when one-minus-fidelity performs better or competitive on these examples.

We therefore start from the partition-level loss

$$d_0(u, c) = -\log F(u, c). \tag{10}$$

To avoid numerical instability at small overlaps, we use the smoothed zero-normalized version

$$d_\varepsilon(u, c) = \log\left(\frac{1+\varepsilon}{F(u, c) + \varepsilon}\right), \qquad \varepsilon > 0. \tag{11}$$

This practical loss stays finite at $F = 0$ and vanishes at $F = 1$. For $\varepsilon = 0$, minimizing $\sum_{p=1}^m -\log F_p$ is equivalent to maximizing $\prod_{p=1}^m F_p$. For $\varepsilon > 0$, the smoothed objective instead ranks pairs by $\prod_{p=1}^m (F_p + \varepsilon)$ and should therefore be viewed as a numerically stable regularized proxy rather than an exactly rank-equivalent reformulation.

Using this loss, the partitioned PQ objective for one sample becomes

$$L(x_i; C) = \sum_{p=1}^{m} \min_{k \in \{1,\ldots,K\}} d_\varepsilon\Big(\psi\big(\phi(x_i^{(p)})\big), c_k^{(p)}\Big). \tag{12}$$

Here $\phi$ is either the identity map or the optional sign-aware encoding. This is the objective optimized by the clustering procedure.

### C. Assignment Rule

Fix a partition $p$ and its current codebook $C^{(p)}$. The assignment for sample $x_i$ in that partition is

$$a_i^{(p)} = \arg\min_{k \in \{1,\ldots,K\}} d_\varepsilon\Big(\psi\big(\phi(x_i^{(p)})\big), c_k^{(p)}\Big). \tag{13}$$

The full PQ code for $x_i$ is then

$$a_i = \big(a_i^{(1)}, a_i^{(2)}, \ldots, a_i^{(m)}\big). \tag{14}$$

In the implementation, the assignment step is evaluated in two modes. The first uses exact overlaps computed classically from normalized vectors. The second uses a shot-based simulator that estimates fidelities via swap-test circuits [9], [10]. The assignment rule itself is unchanged, only the fidelity evaluation differs.

### D. Safeguarded Centroid Update

We consider one partition $p$ and a fixed cluster $I_j = \{i : a_i^{(p)} = j\}$. After applying the representation transform and blockwise normalization, let the assigned states be $u_i \in \mathbb{R}^d$ with $\|u_i\|_2 = 1$. The centroid update solves

$$\begin{aligned} \min_{\|c\|_2=1} f_j(c) &= \sum_{i\in I_j} d_\varepsilon(u_i, c) \\ &= \sum_{i\in I_j} \log\!\left(\frac{1+\varepsilon}{(u_i^\top c)^2+\varepsilon}\right). \end{aligned} \tag{15}$$

Up to the constant $|I_j|\log(1+\varepsilon)$, this is equivalent to minimizing

$$g_j(c) = -\sum_{i\in I_j} \log\big((u_i^\top c)^2+\varepsilon\big). \tag{16}$$

At the current iterate $c_t$, we define the weights

$$w_i(c_t) = \frac{1}{(u_i^\top c_t)^2+\varepsilon}. \tag{17}$$

These weights lead to the quadratic proposal

$$Q(c \mid c_t) = \sum_{i\in I_j} w_i(c_t)(u_i^\top c)^2 = c^\top M(c_t) c, \tag{18}$$

with

$$M(c_t) = \sum_{i\in I_j} w_i(c_t)\, u_i u_i^\top. \tag{19}$$

Since $s \mapsto -\log(s+\varepsilon)$ is convex, $Q(c \mid c_t)$ is not used as a majorization-minimization proposal for $g_j$, but only as a frozen-weight Rayleigh–Ritz proposal. Monotonic decrease is enforced solely by evaluating the true loss in 15 and accepting the proposal only when it lowers that loss.

Maximizing $Q(c \mid c_t)$ on the unit sphere gives the dominant eigenvector of $M(c_t)$. We use this vector as a Rayleigh–Ritz proposal.

As stated above, the proposal is used only when it decreases the true cluster loss in (15). Otherwise, the method falls back to a projected-gradient step with backtracking. The Euclidean gradient of (16) is

$$\nabla g_j(c) = -2\sum_{i\in I_j} \frac{u_i^\top c}{(u_i^\top c)^2+\varepsilon}\, u_i. \tag{20}$$

To remain on the sphere, we project onto the tangent space,

$$P_c(v) = v - (c^\top v)c, \tag{21}$$

and use the retracted update [11]

$$\widetilde{c}(\eta) = \frac{c - \eta\, P_c(\nabla g_j(c))}{\|c - \eta\, P_c(\nabla g_j(c))\|_2}, \tag{22}$$

where the step size $\eta$ is reduced until the true cluster loss decreases.

In practice, the centroid update proceeds as follows. First compute the Rayleigh–Ritz proposal. If that proposal lowers the true loss, accept it. Otherwise take a projected-gradient step with backtracking. Empty clusters retain their previous centroid. This safeguard keeps the update simple while ensuring that every accepted step is checked against the actual smoothed objective.

### E. Practical Stabilization Details

Several implementation details turned out to have practical implications.

First, the same normalized block representation is used throughout training. Initialization, assignment, centroid updates, and diagnostics are all carried out in block-state space rather than mixing raw partition vectors with normalized fidelity-based quantities.

Second, we use the zero-normalized loss in (11) rather than the shifted form $-\log(F+\varepsilon)$. The two induce the same ordering, but $d_\varepsilon(u, c) = 0$ at a perfect match, which makes reported distortions easier to interpret.

Third, convergence checks account for reassignment. In each Lloyd-style iteration [12], we record the objective before the centroid update, after the centroid update under fixed labels, and after reassignment under the new centroids. We stop only once labels are stable and either the relative objective change or the aligned centroid shift falls below threshold.

Finally, centroid shifts are measured in a gauge-aware way. Because the loss depends on $(u^\top c)^2$, the vectors $c$ and $-c$ are equivalent. Before computing shifts, we align each new centroid to maximize its overlap with the previous iterate. This prevents artificial spikes caused only by sign flips.

## IV. Experimental Setup

TABLE I: Datasets, train/test sizes, and random seeds.

| Dataset | Test size | Train sizes | Seeds |
|---|---|---|---|
| Digits | 300 | 60, 120, 200, 300 | 3 |
| Fashion-MNIST $8\times 8$ | 1000 | 100, 200, 300 | 3 |
| Signed-Mirror-64 | 500 | 100, 200 | 5 |

The method is evaluated on two real low-dimensional benchmarks and one synthetic dataset used only for the sign-aware extension. The real datasets are Digits, using the $8\times 8$ optical handwritten digits dataset [13], and Fashion-MNIST [14], whose original $28 \times 28$ images are downsampled here to $8 \times 8$, again yielding 64-dimensional inputs. The synthetic dataset Signed-Mirror-64 is included only to study the sign-aware encoding. Table I summarizes the train/test sizes and the number of random seeds.

For each dataset and random seed, a fixed stratified test split and nested stratified training subsets are constructed. All inputs are $\ell_2$-normalized before applying the PQ pipeline.

Four variants are compared: exact kNN in the normalized original feature space, a classical PQ baseline, an exact-overlap

reference that evaluates fidelities classically from normalized block vectors, and a shot-based swap-test estimator that uses the same assignment rule with simulator-based fidelity estimates.

Accuracy and Recall@10 are reported in the main text. All tabulated values are mean ± standard deviation over the seeds from Table I. Balanced accuracy, macro-F1, and Recall@1 are included in the Repository. Runtime measurements are not part of the evaluation, since in the present simulator setting they are dominated by implementation and execution details.

Unless stated otherwise, the shared quantum configuration for the real-data comparison uses

$$\text{shots} = 2000, \tau = 10^{-2}, \varepsilon = 10^{-3}, m = 8, K = 10. \quad (23)$$

## V. Results and Discussion

### A. Main Benchmark Results

Table II summarizes the results at the largest training size used for each real dataset. On Digits, exact kNN performs best, classical PQ is close behind, and the quantum variants lag by a modest margin. In the shared main-benchmark run, the shot-based model reaches $0.899 \pm 0.022$ accuracy and $0.688 \pm 0.003$ Recall@10, compared with $0.926 \pm 0.019$ and $0.722 \pm 0.007$ for classical PQ.

On Fashion-MNIST $8 \times 8$, the gap is larger. Classical PQ reaches $0.691 \pm 0.021$ accuracy and $0.651 \pm 0.009$ Recall@10, while the shot-based quantum model reaches $0.587 \pm 0.026$ and $0.446 \pm 0.011$. Classical PQ also slightly exceeds exact kNN in accuracy on this split, although exact kNN still yields Recall@10 $= 1.000 \pm 0.000$.

These results suggest that the proposed formulation can support usable codebooks on simpler low-dimensional data, but it is not yet as robust as classical PQ on more difficult datasets.

TABLE II: Main benchmark results at the largest training size used for each real dataset. Values are mean ± standard deviation over the seeds from Table I.

| Dataset | Variant | Accuracy | Recall@10 |
|---|---|---|---|
| Digits ($M = 300$) | exact kNN | $0.934 \pm 0.020$ | $1.000 \pm 0.000$ |
| | classical PQ | $0.926 \pm 0.019$ | $0.722 \pm 0.007$ |
| | shot-based | $0.899 \pm 0.022$ | $0.688 \pm 0.003$ |
| | exact-overlap | $0.894 \pm 0.036$ | $0.677 \pm 0.005$ |
| Fashion $8 \times 8$ ($M = 300$) | classical PQ | $0.691 \pm 0.021$ | $0.651 \pm 0.009$ |
| | exact kNN | $0.683 \pm 0.025$ | $1.000 \pm 0.000$ |
| | shot-based | $0.587 \pm 0.026$ | $0.446 \pm 0.011$ |
| | exact-overlap | $0.515 \pm 0.046$ | $0.319 \pm 0.005$ |

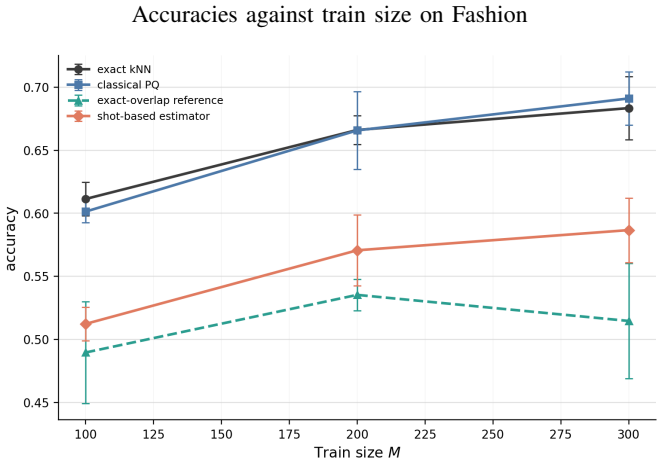


Fig. 2: On Fashion the behaviour is overall compareable to the one on Digits. The main differences remain a significantly larger gap in accuracy between the classical and quantum variants, a decrease in accuracy of quantum exact between $M = 200$ and $M = 300$, and surpassing of classical over exact knn at $M = 300$.

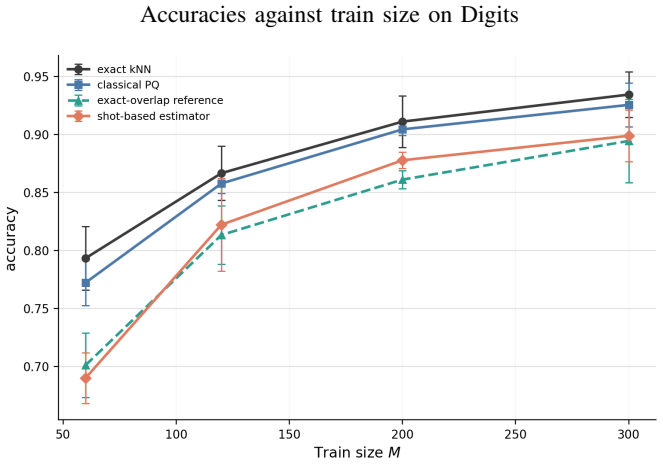


Fig. 1: Although the gap narrows with increasing $M$, the classical and exact variant perform consitently better then their quantum counterparts. While both quantum variants experience a sharp increase in accuracy between $M = 60$ and $M = 120$, their gradient levels with increasing $M$ and at $M = 300$ they perform almost identical. A question for further work might be, if the quantum implementations reach classical accuracies with larger train sizes.

### B. Tolerance, Shots, and the 7000-Shot Regime

The Digits ablations help separate shot effects from optimization effects. At $M = 300$, the shot-based pipeline remains close to the exact-overlap reference across 1000, 2000, and 5000 shots (Table III), with the best mean accuracy in the separate 2000-shot ablation run. Because the shot-based estimator is stochastic and this ablation was executed as a separate plan, the 2000-shot entry in Table III is not expected to match the shared main-benchmark aggregate in Table II exactly. This nevertheless supports the use of 2000 shots and

$\tau = 10^{-2}$ as a reasonable default in the shared real-dataset comparison.

At the same time, the higher-budget Digits sweep shows that the conservative default is not the only useful regime. With 7000 shots, the setting $\tau = 5 \times 10^{-2}$ reaches $0.900 \pm 0.020$ accuracy and $0.688 \pm 0.010$ Recall@10 at $M = 300$, slightly ahead of $0.898 \pm 0.017$ and $0.686 \pm 0.001$ for $\tau = 10^{-2}$.

TABLE III: Digits ablations: shot stability and the 7000-shot regime. Values are mean $\pm$ standard deviation over the seeds from Table I.

| Study | Variant | Accuracy | Recall@10 |
|---|---|---|---|
| $M = 300$ | exact-overlap | $0.894 \pm 0.036$ | $0.677 \pm 0.005$ |
| | shot-based, 1000 | $0.898 \pm 0.005$ | $0.688 \pm 0.006$ |
| | shot-based, 2000 | $0.904 \pm 0.017$ | $0.690 \pm 0.003$ |
| | shot-based, 5000 | $0.900 \pm 0.019$ | $0.684 \pm 0.005$ |
| 7000 shots, $M = 300$ | $\tau = 10^{-2}$ | $0.898 \pm 0.017$ | $0.686 \pm 0.001$ |
| | $\tau = 5 \times 10^{-2}$ | $0.900 \pm 0.020$ | $0.688 \pm 0.010$ |

Looking at the slight improvement of the shot-based estimator over the exact-overlap reference, a plausible explanation is finite-seed variance together with a mild regularization effect induced by stochastically noisy fidelity estimates during both training and assignment. The key observation is that the shot-based pipeline remains consistently close to the exact-overlap reference across the tested shot budgets.

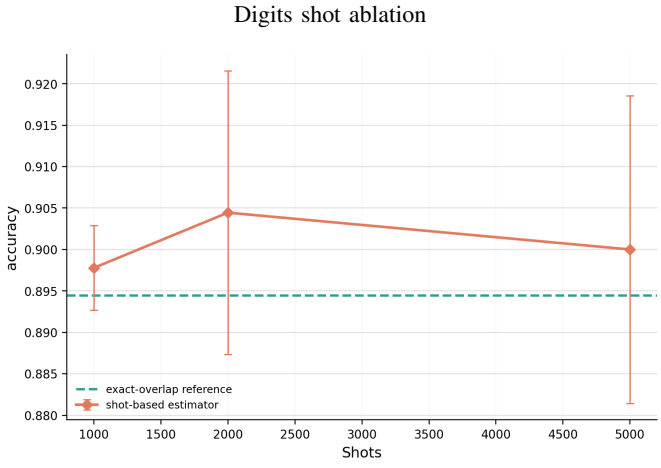


Fig. 3: The shot-based pipeline remains above, but close to the exact-overlap reference across the tested shot counts. Furthermore has the variation of shots only a minor impact on the accuracy with sub-percentage point increases/decreases.

Digits size-variation sweep at 7000 shots.

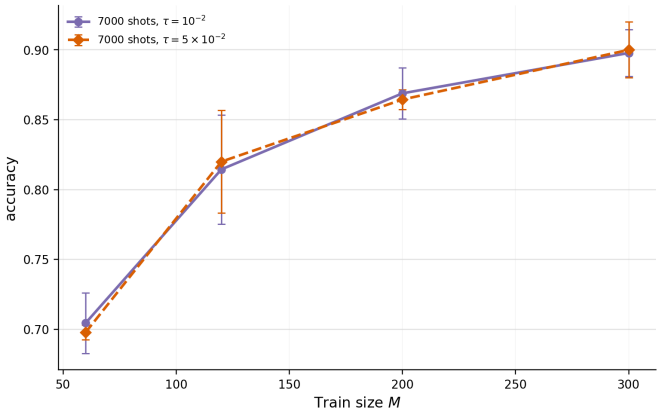


Fig. 4: Overall the 7000 shot regime shows the same behaviour as the other ones, with a sharp increase at small $M$ and continouus, but slower increase at larger $M$. The higher-tolerance setting $\tau = 5 \times 10^{-2}$ slightly improves the result at the largest training size, which might be an interesting parameter to tune under higher shots, because this decreases the amount of iterations per $k$-means.

### C. Additional Studies

Table IV reports two additional analyses. The first compares additive log-fidelity with one-minus-fidelity on Digits in exact-overlap reference mode. One-minus-fidelity is slightly stronger in this setting, reaching $0.906 \pm 0.020$ accuracy and $0.694 \pm 0.013$ Recall@10 versus $0.894 \pm 0.036$ and $0.677 \pm 0.005$ for log-fidelity.

The second study examines the sign ambiguity of raw fidelity on Signed-Mirror-64. Because fidelity depends on the squared inner product, sign-flipped inputs can become indistinguishable. That issue is visible in the raw exact-overlap result, which reaches only $0.493 \pm 0.020$ accuracy. Once the sign-aware encoding is used, accuracy rises to $1.000 \pm 0.000$, matching exact kNN on this metric, showing that the sign ambiguity can be removed by this step.

TABLE IV: Additional studies: metric comparison on Digits and sign-aware encoding on Signed-Mirror-64. Values are mean $\pm$ standard deviation over the seeds from Table I.

| Study | Variant | Accuracy | Recall@10 |
|---|---|---|---|
| Digits, exact mode | log-fidelity | $0.894 \pm 0.036$ | $0.677 \pm 0.005$ |
| | one-minus-fidelity | $0.906 \pm 0.020$ | $0.694 \pm 0.013$ |
| Signed-Mirror-64 | exact kNN | $1.000 \pm 0.000$ | $1.000 \pm 0.000$ |
| | exact-overlap (raw) | $0.493 \pm 0.020$ | $0.410 \pm 0.074$ |
| | exact-overlap (sign-aware) | $1.000 \pm 0.000$ | $0.789 \pm 0.161$ |

Overall, the empirical picture is mixed but internally consistent. The proposed formulation works well on Digits, deteriorates on a harder real dataset, and shows mild gains in a higher-shot regime. The present evaluation is still limited to small low-dimensional benchmarks and relies on simulator-based quantum estimates. Therefore these results serve as a proof-of-concept and further investigations are needed.

## VI. Conclusion

This paper introduced an additive log-fidelity quantum k-means formulation for hybrid product quantization and evaluated it in a PQ-kNN pipeline. The main methodological point is that a fidelity-based clustering objective can be rewritten to match the additive block structure required by PQ and trained stably enough to yield reasonable codebooks in practice.

The empirical evidence is strongest on Digits, where the shot-based variant remains close to the classical PQ baseline and where a slightly stronger 7000-shot regime can already be identified. Results on downsampled Fashion-MNIST are clearly weaker, which shows that the approach is not yet robust on harder datasets. The synthetic signed benchmark further hands a solution for the genuine sign ambiguity.

A natural next step is to test the formulation on broader ANN benchmarks, add matched sign-aware classical baselines, and examine whether the higher-shot Digits gains persist on more challenging data. It will also be useful to revisit the comparison with one-minus-fidelity under the same broader evaluation protocol and examine the approach more rigorously from a mathematical standpoint.


## Acknowledgment

OpenAI ChatGPT was used to assist with code generation for parts of the repository accompanying this paper, especially for evaluation scripts and experiment automation scripts. All algorithmic content, experimental configuration, correctness checks, and the final code used to produce the reported results were reviewed and validated by the authors. Furthermore this AI system was used throughout each part of the paper for grammar and general language checks and help with textual formulations.


## References


[1] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[2] T. Ge, K. He, Q. Ke, and J. Sun, "Optimized product quantization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 4, pp. 744–755, 2014.

[3] E. Aïmeur, G. Brassard, and S. Gambs, "Quantum clustering algorithms," in *Proceedings of the 24th International Conference on Machine Learning*, 2007, pp. 1–8.

[4] S. Lloyd, M. Mohseni, and P. Rebentrost, "Quantum algorithms for supervised and unsupervised machine learning," *arXiv preprint arXiv:1307.0411*, 2013.

[5] I. Kerenidis, J. Landman, A. Luongo, and A. Prakash, "q-means: A quantum algorithm for unsupervised machine learning," in *Advances in Neural Information Processing Systems 32*, 2019.

[6] A. Cornelissen, J. F. Doriguello, A. Luongo, and E. Tang, "Do you know what *q*-means?" *arXiv preprint arXiv:2308.09701*, 2023.

[7] T. Chen, A. Ray, A. Seshadri, D. Herman, B. Bach, P. Deshpande, A. Som, N. Kumar, and M. Pistoia, "Provably faster randomized and quantum algorithms for *k*-means clustering via uniform sampling," *arXiv preprint arXiv:2504.20982*, 2025.

[8] R. Jozsa, "Fidelity for mixed quantum states," *Journal of Modern Optics*, vol. 41, no. 12, pp. 2315–2323, 1994.

[9] A. Barenco, A. Berthiaume, D. Deutsch, A. Ekert, R. Jozsa, and C. Macchiavello, "Stabilization of quantum computations by symmetrization," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1541–1557, 1997.

[10] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, "Quantum fingerprinting," *Physical Review Letters*, vol. 87, no. 16, p. 167902, 2001.

[11] P.-A. Absil, R. Mahony, and R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*. Princeton, NJ: Princeton University Press, 2008.

[12] S. P. Lloyd, "Least squares quantization in pcm," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[13] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits," UCI Machine Learning Repository, 1998, dataset. [Online]. Available: https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits

[14] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.